

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

CLI To-Do List Manager - Technical Specification & Architecture Document	3
1. Executive Summary & Architecture Overview	3
1.1 Executive Brief	3
1.2 Maturity Assessment	3
1.3 Technical Stack	3
1.4 Architectural Constraints	3
1.5 Critical Dependencies	3
2. Architecture Workflows	4
2.1 Project Implementation Roadmap & Traceability	5
2.2 CLI Task Management Workflow	5
2.3 CLI Command Execution Sequence	6
3. Detailed Technical Specifications & Business Rules	7
3.1 Requirements Traceability	7
3.2 Security Rules	9
3.3 Data Models	9
4. Project Governance & Structural Gaps	9
4.1 Structural Gaps	9
4.2 Remediation & Workflow	10
5. Technical & Domain Glossary (Terminology Reference)	10

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a Python-based command-line application designed for task lifecycle management using a local JSON storage pattern. It implements a layered architecture comprising a CLI dispatch layer, a service layer for business logic, and a storage layer for file I/O. The system focuses on core CRUD operations: adding tasks with sequential IDs, listing tasks in human-readable or JSON formats, and managing task completion and removal.

1.2 Maturity Assessment

The project is READY for execution. While there are medium-severity gaps regarding formal acceptance criteria and low-severity omissions of security/performance constraints, the task graph is structurally complete with a high health index. The execution path is clearly defined through a blocking foundational phase followed by independent user stories, ensuring a stable implementation sequence.

1.3 Technical Stack

- Python
- pyproject.toml

1.4 Architectural Constraints

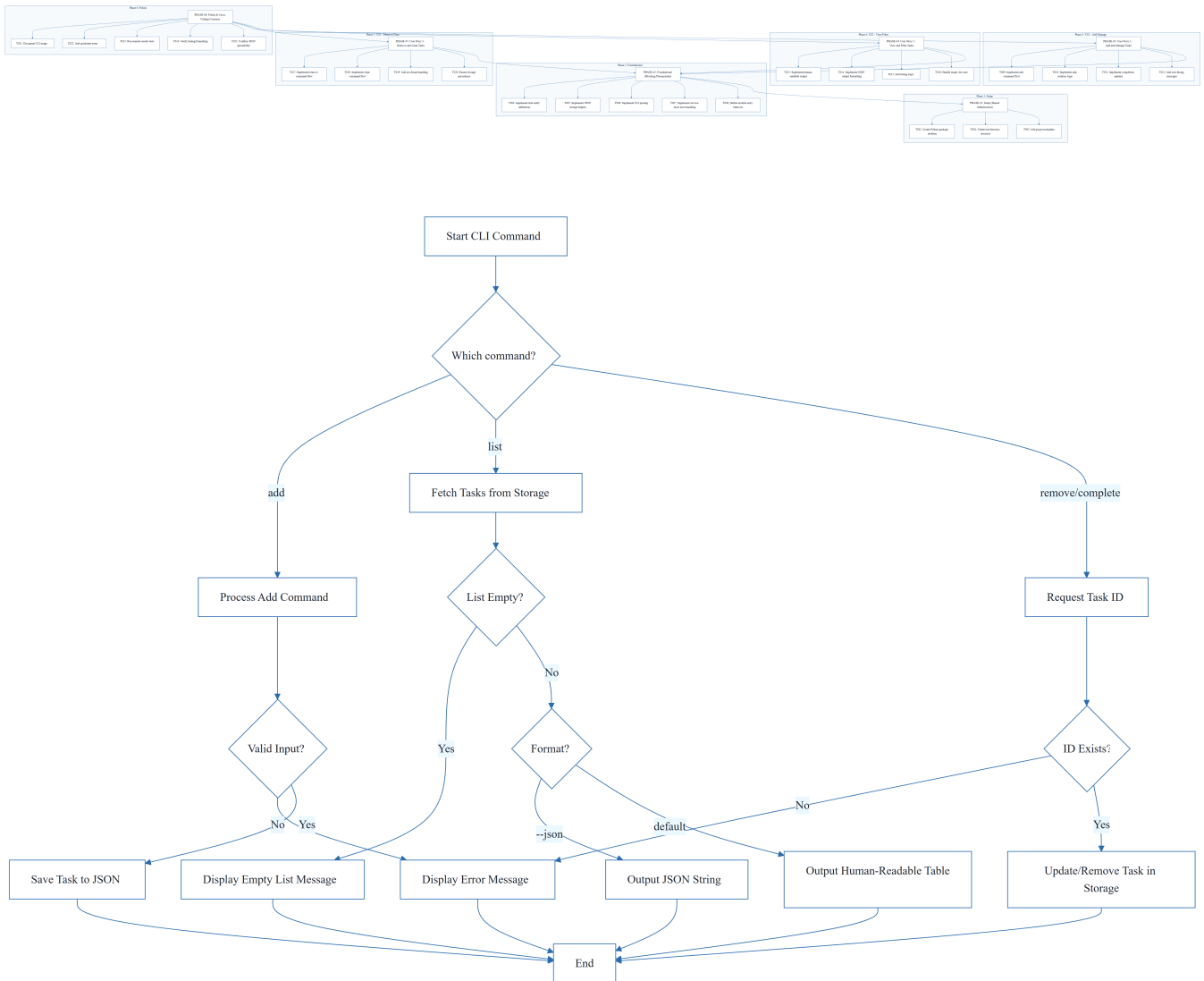
- Sequential ID assignment for task creation.
- Strict JSON storage format for persistence.
- Mandatory completion of Foundational Phase (PHASE-02) before any User Story implementation.
- Human-readable and JSON output modes for listing.
- Manual smoke test validation for all core commands (add, list, complete, remove, clear).
- Linting and formatting verification required for all source files in `src/todo_manager/`.

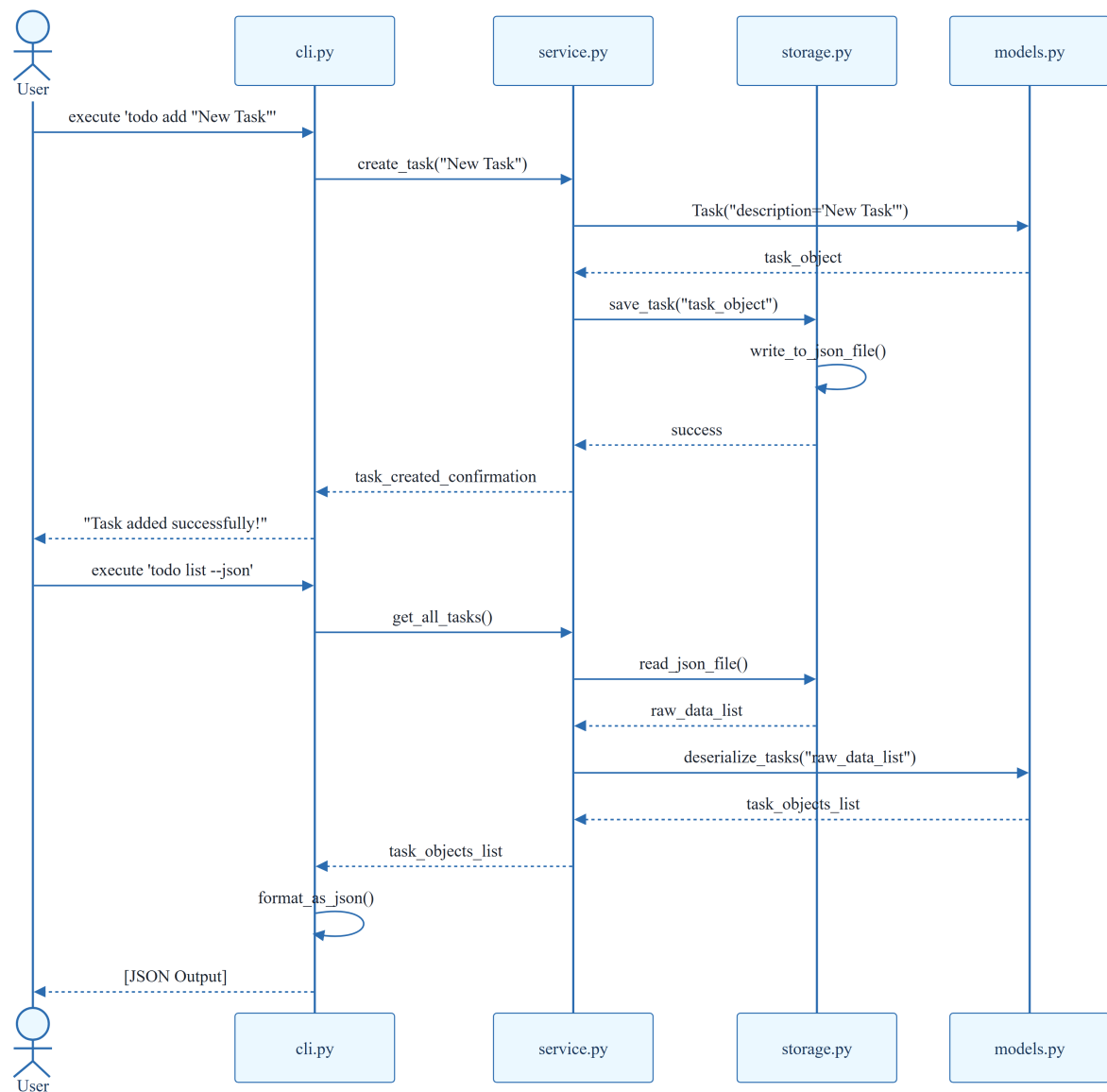
1.5 Critical Dependencies

- Local file system access for `~/ .todos .json` storage.
- Python package skeleton integrity (`src/todo_manager/`).
- Sequential dependency: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.

- Internal data integrity: Task entity serialization helpers must precede CLI wiring.
- Internal data integrity: Service-layer logic must precede output formatting.

2. Architecture Workflows

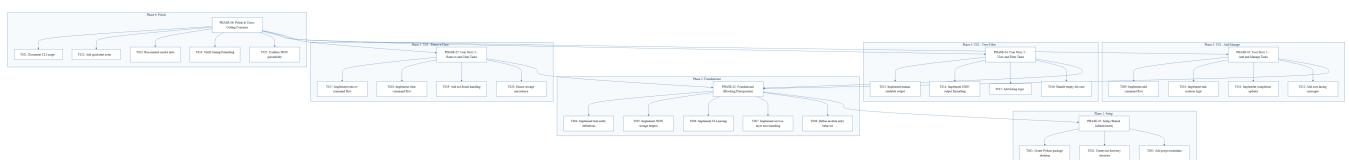




& Visual Diagrams

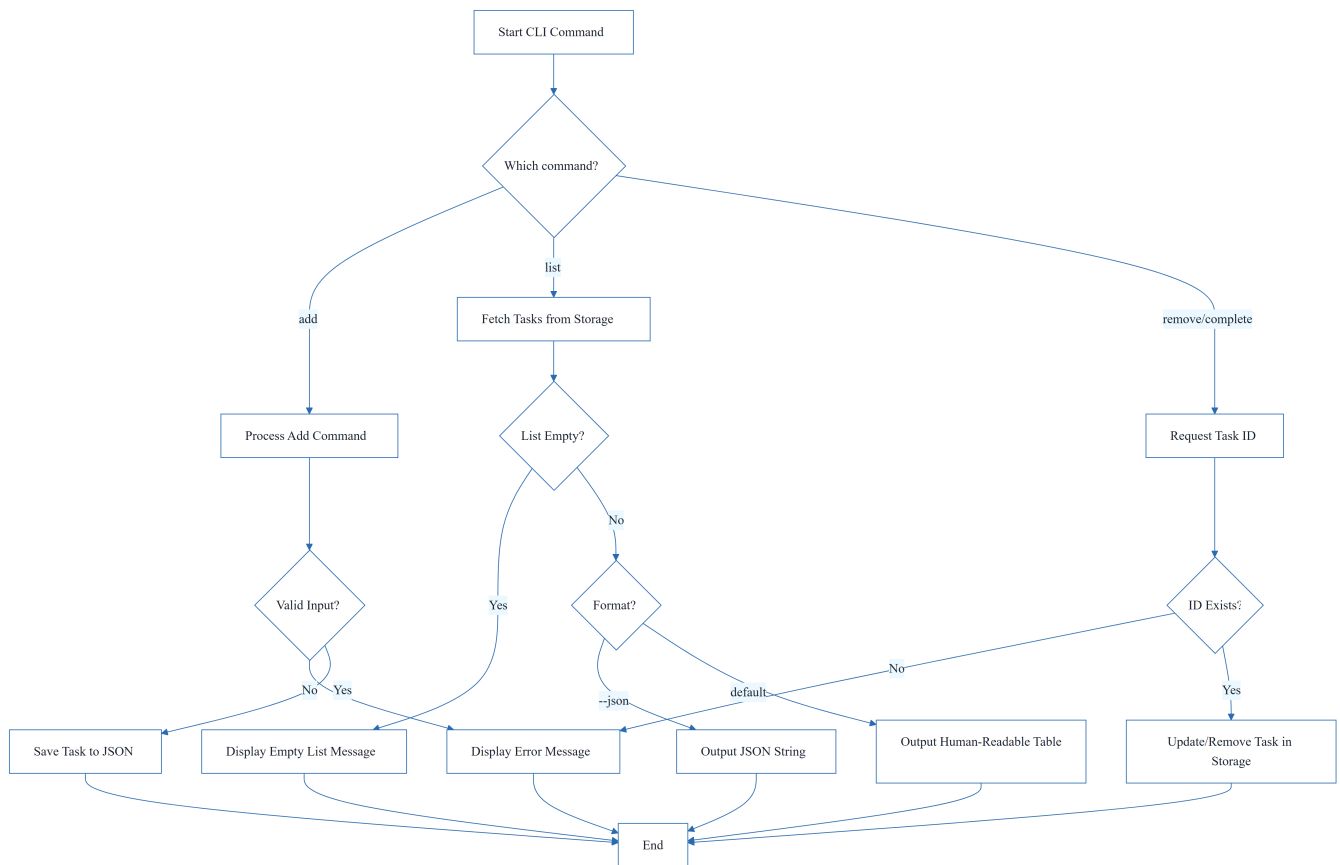
2.1 Project Implementation Roadmap & Traceability

Visualizes the dependency chain between project phases and the specific tasks contained within each phase, ensuring strict traceability to original identifiers.



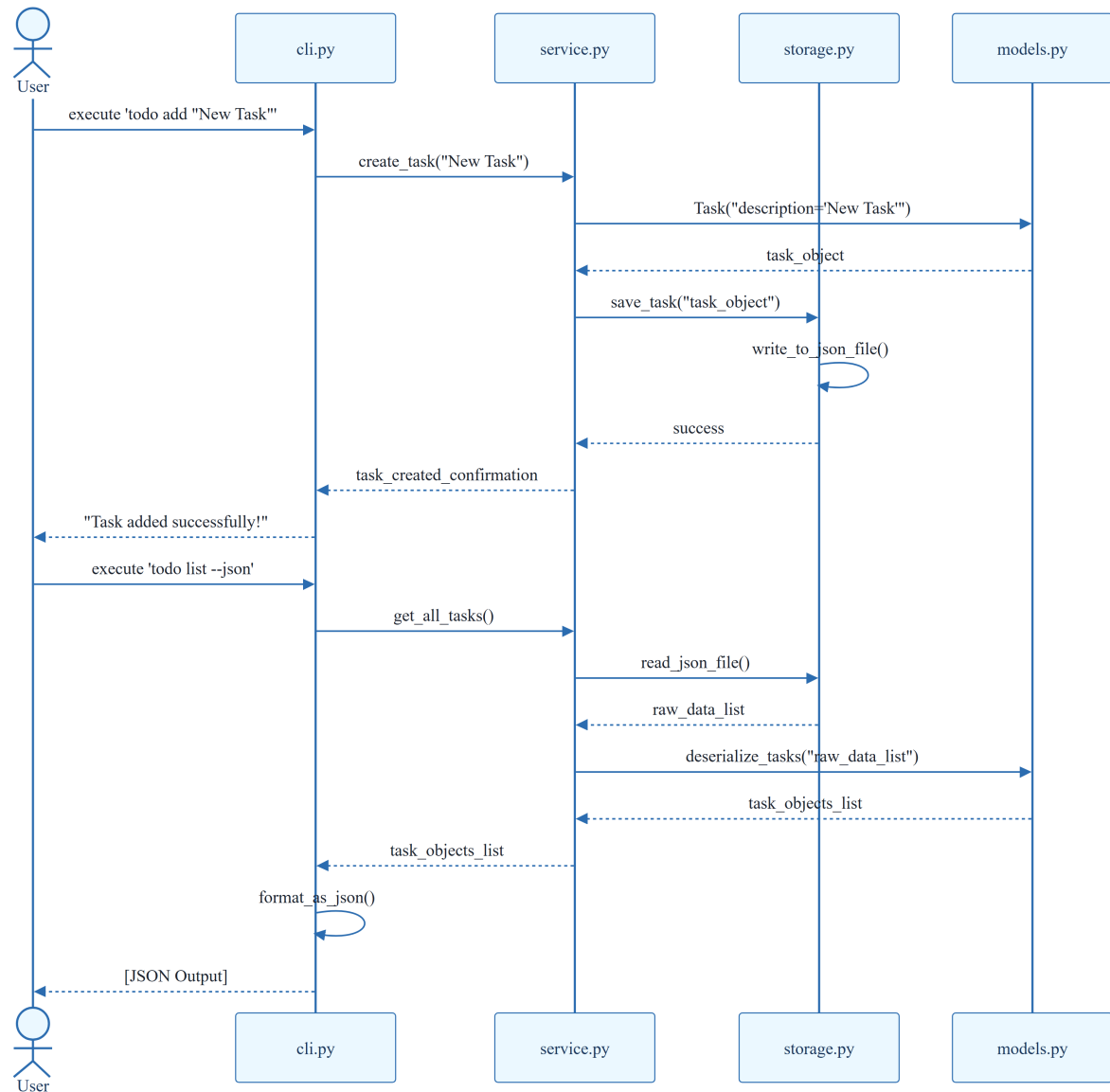
2.2 CLI Task Management Workflow

Models the business logic for task operations, including decision points for input validation and output formatting.



2.3 CLI Command Execution Sequence

Illustrates the interaction between the CLI layer, the Service layer, and the Storage layer for a typical task operation.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Requirement / Task Description	Source Section	Attributes / Notes
T001	Create the Python package skeleton	Phase 1: Setup	Files: src/todo_manager/
T002	Create the test directory structure	Phase 1: Setup	tests/unit/, tests/integration/
T003	Add project metadata and tooling entry points in pyproject.toml	Phase 1: Setup	Parallel execution enabled

ID	Requirement / Task Description	Source Section	Attributes / Notes
T004	Implement task entity definitions and serialization helpers	Phase 2: Foundational	src/todo_manager/models.py
T005	Implement JSON storage path resolution and file I/O helpers	Phase 2: Foundational	src/todo_manager/storage.py
T006	Implement shared command-line parsing and top-level dispatch	Phase 2: Foundational	Parallel; src/todo_manager/cli.py
T007	Implement shared service-layer error handling and task collection utilities	Phase 2: Foundational	src/todo_manager/service.py
T008	Define module entry behavior for python -m todo_manager	Phase 2: Foundational	src/todo_manager/__main__.py
T009	Implement the add command flow	Implementation for US1	Story: US1
T010	Implement task creation, sequential ID assignment, and created_at population	Implementation for US1	Story: US1
T011	Implement completion updates for existing tasks	Implementation for US1	Story: US1
T012	Add user-facing success and error messages for add and complete operations	Implementation for US1	Story: US1
T013	Implement human-readable task listing output	Implementation for US2	Story: US2
T014	Implement --json output formatting	Implementation for US2	Story: US2
T015	Add listing logic that preserves task order and status fields	Implementation for US2	Story: US2
T016	Handle the empty-list case with a clear message	Implementation for US2	Story: US2
T017	Implement the remove command flow	Implementation for US3	Story: US3
T018	Implement the clear command flow for removing completed tasks	Implementation for US3	Story: US3
T019	Add not-found handling for invalid task IDs	Implementation for US3	Story: US3

ID	Requirement / Task Description	Source Section	Attributes / Notes
T020	Ensure storage writes persist removals and clears safely	Implementation for US3	Story: US3
T021	Document the CLI usage and storage behavior in README.md	Phase 6: Polish	Parallel execution enabled
T022	Add quickstart verification notes and examples	Phase 6: Polish	Parallel execution enabled
T023	Run a manual smoke test of add, list, complete, remove, and clear	Phase 6: Polish	Type: smoke_test
T024	Verify linting and formatting expectations	Phase 6: Polish	src/todo_manager/
T025	Confirm the generated JSON output remains parseable	Phase 6: Polish	Validation of todo list --json

3.2 Security Rules

- No specific security constraints were defined for this CLI tool.

3.3 Data Models

- **Task Entity:** Defined in `src/todo_manager/models.py`. Includes sequential ID, description, completion status, and `created_at` timestamp.
- **Persistence:** JSON file located at `~/todos.json`.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Missing Section	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' are mentioned for user stories, formal acceptance criteria for each task are missing.
Security & Performance Constraints	LOW	No specific security or performance constraints were defined for this CLI tool.
Open Questions & Uncertainties	LOW	The document appears complete from a task perspective; no open questions were listed.

4.2 Remediation & Workflow

The project follows an incremental delivery model:

1. **Setup $\rightarrow$ Foundational:** Establish the core I/O and model layers.
2. **User Story Implementation:** Execute US1 (MVP), then US2, then US3.
3. **Validation:** Each story is validated independently before proceeding.
4. **Polish:** Final smoke tests and documentation.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-02	A synchronization gate ensuring all blocking infrastructure is verified before parallel feature development commences.
Foundational	TECHNICAL_STACK	PHASE-02	The critical infrastructure layer containing shared serialization and I/O logic that blocks all subsequent user story implementation.
Goal	BUSINESS_DOMAIN	PHASE-03	The primary functional objective a user must achieve within a specific feature set.
ID	BUSINESS_DOMAIN	T010	A sequential alphanumeric token assigned to each entry to ensure unique reference and retrieval.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage and scriptable output.
MVP	BUSINESS_DOMAIN	PHASE-03	The minimum viable product consisting of the most critical feature set required for initial validation.
Organization	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by their corresponding user story to ensure independent testability.

Term	Category	Context Anchor	Project Definition
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of design and contract documents required before implementation begins.
README	TECHNICAL_STACK	T021	The primary documentation file detailing usage and storage behavior.
Setup	TECHNICAL_STACK	PHASE-01	The initial phase focused on package skeleton creation and tooling configuration.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for handling empty collection states with user-facing messages.
Tests	TECHNICAL_STACK	T002	The verification suite comprising unit and integration directories to validate system behavior.
population in	BUSINESS_DOMAIN	T010	The process of assigning a timestamp to the creation field during entry instantiation.
todo clear	BUSINESS_DOMAIN	T018	The operation that purges all entries marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operation that retrieves and displays all stored entries in either human-readable or machine-parseable formats.
■■ CRITICAL	TECHNICAL_STACK	PHASE-02	A high-priority constraint indicating that subsequent work is strictly blocked until the current phase is finalized.